arXiv:2610.34907v1 [cs.DC] 26 Oct 2026

When and Why to Use TMA: A Workload-Grounded Characterization of Tensor Memory Acceleration on NVIDIA Hopper GPUs

Christin David Bose, Cesar Avalos Baddouh, Junrui Pan, Timothy G. Rogers
Purdue University, West Lafayette, IN, USA
{chris241, cavalosb, pan251, timrogers}@purdue.edu

Abstract

The Tensor Memory Accelerator (TMA), introduced on NVIDIA Hopper, is used pervasively in production GPU kernels yet its usage is conspicuously selective: expert kernel developers in CUTLASS, FlashAttention-3, and FlashInfer apply TMA to some tensor operands within a kernel but fall back to cp.async, direct global loads, or distributed shared memory (DSM) for others. Vendor documentation describes the API, and recent microbenchmarking work characterizes TMA throughput in isolation, but neither answers the question practitioners actually face: given a memory operand in a kernel, should it use TMA, and if so, with or without cluster multicast? We address this gap with a workload-grounded characterization study on H100. We catalog the TMA, multicast, DSM, and cp.async choices made inside FlashAttention-3 forward, isolate the discriminating features through targeted microbenchmarks (transfer-size crossover, multicast scaling), and produce rules of thumb for the load-mechanism choice on Hopper. We further compare TMA multicast against DSM as competing cluster-broadcast primitives, and isolate the FlashAttention-3 paged-key-value (KV) TMA-vs-cp.async dispatch on H100 across page size, datatype, model shape, and execution phase. Our results yield concrete, predictive guidance on when TMA pays off and when multicast helps or hurts on Hopper, and we extend the analysis to server Blackwell (SM100) via a CUTLASS GEMM study on B200 that shows the qualitative dispatch rules carry across generations while absolute crossovers shift with tensor-core and bandwidth scaling.

1. Introduction

The Tensor Memory Accelerator (TMA) is the headline memory-system addition of NVIDIA's Hopper architecture [16]. By issuing a single hardware-managed bulk asynchronous copy from global to shared memory, indexed through a precomputed tensor descriptor, TMA frees warps from generating per-thread addresses and overlaps data movement with tensor-core compute (Figure 1). It also exposes two features that have no equivalent in the prior `cp.async` path: hardware-supported swizzling for bank-conflict-free SMEM layouts feeding tensor cores, and a *multicast* mode that broadcasts the same tile to multiple SMs in a thread-block cluster with a single issue and a single L2 read.

TMA is used heavily in production. CUTLASS [28] dispatches its SM90 GEMM mainloops through TMA for the operand matrices; FlashAttention-3 [27] routes Q, K, and V through TMA on its prefill path; FlashInfer [32] uses TMA for contiguous KV. Public write-ups attribute much of the resulting SM90 uplift to TMA [2, 5].

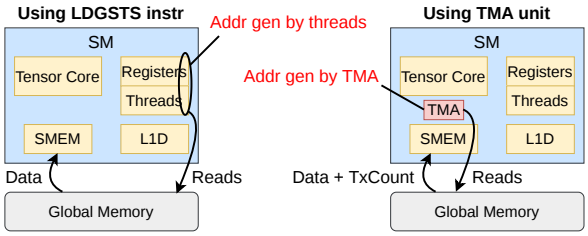


Figure 1: High-level view of the `cp.async` and TMA paths on Hopper

And yet TMA is not used for everything. Expert kernel code selectively avoids TMA in regimes where one might naively expect it to win. Three concrete examples motivate this paper.

First, FlashAttention-3 contains an in-tree dispatch heuristic that disables the TMA-paged-KV path when per-tile work falls below a compute-bound threshold [31]. The heuristic was shipped on empirical grounds alone, without an articulated mechanism. A companion issue [29] reports a related TMA overhead pattern in deployed batch decode.

Second, FlashInfer [32] explicitly states that TMA is used *only* for contiguous KV on Hopper, and falls back to `cp.async` for paged KV. PagedAttention [7] and vAttention [26] treat the KV page-size knob as a purely allocator-level tradeoff (small pages waste fewer tokens at the tail; large pages amortize lookup cost), but the choice of page size also dictates whether TMA is even usable for the KV load (FA3 requires the page size to be a multiple of the kernel's K-tile), and operators of vLLM [30] and TensorRT-LLM [23] currently guess the page size without quantitative guidance on how much TMA bandwidth they leave on the cp.async fallback.

Third, the choice between *TMA multicast* and *distributed shared memory* (DSM), the two hardware mechanisms Hopper provides for cluster-wide tile sharing, is unsettled even among expert codebases. CUTLASS GEMMs broadcast operand tiles across cluster CTAs via TMA multicast [28], while community reports [24, 25] flag regimes in which multicast underperforms independent unicast TMAs from

the same cluster. Both hardware paths solve the same logical problem (get one Global Memory (GMEM) tile to N SMs in a cluster), but no published work characterizes when one beats the other, and DSM as an alternative cluster-broadcast primitive has received no systematic comparison at all.

These three observations are not independent oddities; they share a common cause. TMA optimizes for a specific access regime (large, multi-dimensional, well-aligned tiles whose descriptor cost amortizes over many issues) and degrades, sometimes sharply, outside that regime. Production code encodes this fact in dispatch heuristics written by experts, but the heuristics are workload-specific, underdocumented, and not always articulated by their authors. NVIDIA's Hopper Tuning Guide [20] describes the API but does not provide crossover points; recent academic microbenchmarking work characterizes TMA throughput and latency in isolation, or the cp.async path in isolation, but does not address the comparative question [6, 9–13].

The questions this paper answers. We frame the gap as three concrete questions a kernel developer can act on:

1) *When does TMA outperform* `cp.async`*, and by how much?* What are the crossover points in transfer size and bytes-in-flight per SM, and how do they manifest in production attention across page size, datatype, model shape, and execution phase?
2) *When does TMA multicast pay off, and when does it hurt?* How does the answer depend on cluster size and tile size, and is the L2 Request Coalescer (LRC)-deduplicated redundant-unicast path a better choice in the streaming regime?
3) *Which cluster-broadcast primitive should one pick?* How does TMA multicast compare head-to-head against distributed-shared-memory (DSM) broadcast when the DRAM cost is matched?

Approach. We answer these questions through a workload-grounded characterization study. We first *catalog* the memory-mechanism choice (TMA, TMA multicast, DSM, `cp.async`) for every operand of one production kernel: FlashAttention-3 forward, on prefill, chunked prefill, and decode paths across dense multi-head attention (MHA), grouped-query attention (GQA), and multi-head latent attention (MLA) shapes. From the catalog we form *hypotheses* for the discriminating features (dimensionality, transfer size, reuse, multicast applicability, multicast-vs-DSM preconditions). We then test these hypotheses with targeted microbenchmarks on H100 (SM90) that sweep the relevant axes in isolation and measure both wall-clock time and the underlying GPU metrics with NVIDIA Nsight Compute (NCU) [18].

A cross-cutting case study runs through the research questions: the FlashAttention-3 [27] paged-KV TMA-vs-`cp.async` dispatch on H100. FA3's modular dispatch means the TMA-paged path engages only at page sizes that are multiples of the kernel's K-tile, while every other commonly deployed page size falls back to `cp.async`, a configuration whose performance impact operator-facing tools have not yet measured. Sweeping page size across dense MHA (Llama-3-8B [14]), GQA (Llama-3-70B), and MLA (DeepSeek-V2 [4], with FlashMLA [8] as the production reference) directly answers a question vLLM/SGLang/TensorRT-LLM operators currently guess at, and isolates whether FA3's compute-bound gate is empirically necessary across the deployed model.

Contributions. We make three contributions, mapped one-to-one to the research questions above:

- *(Q1)* A workload-grounded characterization of when TMA outperforms `cp.async` on Hopper, combining a microbench crossover study with a controlled sweep comparing TMA-paged against forced `cp.async`-paged on FA3 across page size, datatype, model shape, and execution phase.
- *(Q2)* A characterization of TMA multicast against the LRC-deduplicated redundant-unicast path as competing intra-cluster fan-out primitives, swept across cluster size and tile size, identifying the regime in which redundant unicast is faster end-to-end and the regime in which multicast's L2 cleanliness is essential.
- *(Q3)* A head-to-head comparison of TMA multicast against DSM broadcast at matched DRAM cost, across cluster size and tile size on H100, showing when each primitive is the right choice for the regular broadcast pattern.

The remainder of the paper is organized as follows. Section II provides background on TMA, `cp.async`, thread-block clusters, multicast, and DSM on Hopper. Section III describes the experimental methodology and workload corpus. Section IV reports the characterization results, including the FA3 paged-KV TMA-vs-`cp.async` case study and the multicast-vs-redundant-unicast comparison. Section V extends the characterization to server Blackwell (SM100) via a CUTLASS GEMM study on B200. Section VI discusses related work. Section VII concludes.

2. Background

This section reviews the GPU memory mechanisms that this paper characterizes and compares. We focus on the level of detail required to interpret the experiments in Section 4. We assume basic familiarity with the CUDA execution model.

2.1. The asynchronous-pipeline ideal on Hopper

The performance ceiling of a Hopper tensor-core kernel is set by how completely the warp groups doing math overlap the warps doing memory movement. CUTLASS [28], FlashAttention-3 [27], and FlashInfer [32] all build on the same template: one or more *producer* warps stream tiles from global memory (GMEM) into shared memory (SMEM) in software pipelined stages, and one or more *consumer* warp groups issue asynchronous Warp Group MMA (WGMMA) tensor-core operations out of those SMEM stages. Producers and consumers synchronize through `mbarrier` objects in SMEM. The kernel hits peak when the

Algorithm 1 `cp.async` enabled producer/consumer pipeline.

```
1: if producer warpgroup then
2:   for tile ∈ tiles do
3:     b ← tile mod 2
4:     wait_until_buffer_free(b)
5:     gmem_ptr ← &global[tile][thread_slice]
6:     smem_ptr ← &smem[b][thread_slice]
7:     cp.async.shared.global(smem_ptr, gmem_ptr, B_th)
8:     cp.async.commit_group()
9:     cp.async.wait_group(0)
10:    signal_buffer_ready(b)
11:  end for
12: end if
13: if consumer warpgroup then
14:  for tile ∈ tiles do
15:    b ← tile mod 2
16:    wait_until_buffer_ready(b)
17:    compute_on_tile(smem[b])
18:    signal_buffer_free(b)
19:  end for
20: end if
```

Algorithm 2 TMA enabled producer/consumer pipeline.

```
1: if producer warpgroup then
2:   for tile ∈ tiles do
3:     b ← tile mod 2
4:     wait_until_buffer_free(b)
5:     coords ← tensor_tile_coords(tile)
6:     tma.load(dst_smem=smem[b],      tensor_map=desc,
       coords=coords, completion_barrier=ready[b])
7:   end for
8: end if
9: if consumer warpgroup then
10:  for tile ∈ tiles do
11:    b ← tile mod 2
12:    wait_until_buffer_ready(b)
13:    compute_on_tile(smem[b])
14:    signal_buffer_free(b)
15:  end for
16: end if
```

producer can keep N tile stages full while the consumers drain them at WGMMA throughput.

Two features of Hopper directly raise this ceiling: the Tensor Memory Accelerator and thread-block clusters. The choice of *which* memory primitive the producer warps use to fill SMEM (TMA, TMA with multicast, distributed shared memory (DSM), or the older `cp.async` (LDGSTS) path) is the choice this paper characterizes. Algorithms 1 and 2 sketch the same producer/consumer pipeline filled by `cp.async` versus TMA: the TMA variant replaces every producer thread's address arithmetic and per-thread issue stream with a single `cp.async.bulk.tensor` from one elected thread against a precomputed tensor-map descriptor.

Algorithms 1 and 2 contrast the two paths at the producer-warp level. The structural difference is where address generation lives: `cp.async` pushes per-thread address arithmetic into the warp's issue stream every iteration, while TMA hoists the address computation into a one-time descriptor build and reduces the per-iteration cost to a single bulk-issue plus a barrier wait. This distinction (per-thread vs. single-thread issue, per-tile vs. per-tensor address logic) is the structural source of every quantitative crossover we report in Section 4.

2.2. `cp.async` (LDGSTS): the pre-Hopper baseline

`cp.async`, exposed at the SASS level as LDGSTS, was introduced on Ampere (SM80) [15] and remains the fallback path on Hopper. Each thread issues its own load with a per-thread address. The instruction is asynchronous in that the issuing thread does not stall on the load; completion is established with an explicit `cp.async.wait_group` (paired with a CTA barrier) before the SMEM contents are consumed. `cp.async` loads go through the L1/tex pipe and SMEM is filled per-thread.

Two properties of `cp.async` matter for the comparison: (i) the address generation cost scales linearly with the number of participating threads, so issuing a tile load consumes warp issue-slot bandwidth and burns instruction-cache pressure; and (ii) because each thread writes its own SMEM bytes, achieving bank-conflict-free SMEM layouts for downstream WGMMA requires *software* swizzling in the address calculation. Li et al. [9, 10] characterize the `cp.async` (Async Memcpy) path itself on A100 and H100, reporting the regimes in which it overlaps usefully with compute; the Hopper microbenchmarking work of Luo et al. [12] reports the per-instruction overhead and L1-pipe behavior in detail. Neither line of work compares `cp.async` against TMA on the same operand, which is the comparison this paper makes.

2.3. Tensor Memory Accelerator (TMA)

TMA, exposed in PTX as the `cp.async.bulk.tensor` family [21], replaces the per-thread address-generation model of `cp.async` with a hardware-managed bulk transfer. A single thread in a CTA issues one TMA instruction, and a dedicated hardware unit (the TMA engine, one per SM) walks a precomputed tensor descriptor and streams a multi-dimensional tile from GMEM into SMEM. Three aspects of the design drive the comparisons in this paper.

Tensor descriptors. A TMA load is parameterized by a `CUtensorMap` object built on the host (or on-device via `tensormap.replace`) that encodes the GMEM base pointer, per-dimension stride and extent, tile shape, fill mode, and swizzle pattern. The descriptor is shared across all TMA issues that index into the same logical tensor. Descriptor construction is a one-time per-tensor setup performed outside the kernel's inner loops; each TMA *issue* against a built descriptor then adds a ∼170-cycle per-access overhead before the bulk transfer begins [12]. The descriptor cost is therefore a fixed overhead amortized over all subsequent issues against the same tensor, and the per-issue cost is amortized over the bulk transfer size: tiny transfers or low descriptor reuse leave both costs unamortized.

Shape constraints. A TMA tensor descriptor supports 1-D through 5-D access. Each non-stride dimension is

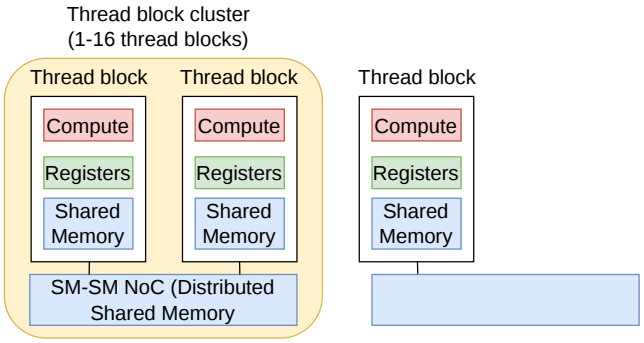


Figure 2: Illustration of a thread block cluster

capped at 256 elements, which makes 1-D TMA practically limited to about 2 KB per issue [12]. Higher-rank descriptors lift the per-issue size but introduce per-rank stride and box-shape constraints; in particular, the descriptor's leading (x) dimension extracts higher throughput than the trailing (y, z) dimensions, an asymmetry first reported in [12]. TMA addresses must be 16-byte aligned and the descriptor stride along the inner dimension must be a multiple of 16 bytes; misalignment is a hard correctness requirement, not a performance penalty.

SMEM path and hardware swizzle. TMA bypasses L1 entirely, reads through L2, and writes directly to SMEM [12]. The descriptor specifies one of four hardware swizzle patterns (none, 32B, 64B, 128B) which the TMA engine applies during the write so that the resulting SMEM layout is bank-conflict-free for WGMMA consumption.

Completion. TMA completion is signaled through a synchronization primitive such as an `mbarrier` [22] in SMEM that the issuing thread arms with the expected byte count; consumer warps wait on the barrier before reading the tile. The same mbarrier infrastructure is shared with tensor core accumulator tracking, which is what makes the producer/consumer pipeline efficient.

2.4. Thread-block clusters

Hopper introduced the thread-block cluster (Figure 2), a new grouping above the Cooperative Thread Array (CTA) in the execution hierarchy [16]. A cluster is a set of up to 16 CTAs co-scheduled onto adjacent SMs in a Graphics Processing Cluster (GPC). Cluster CTAs can synchronize through cluster-wide barriers, and, critically for this paper, they can address each other's SMEM via the *Distributed Shared Memory* (DSM) Network-on-Chip (NoC) described below. Clusters are the unit on which both TMA multicast and DSM operate; outside a cluster, neither mechanism is available. NVIDIA H100 [16] GPUs have also introduced an L2 Request Coalescer (LRC) that processes incoming requests for L2 and tries to coalesce read requests before forwarding them to the L2 cache [18].

2.5. TMA multicast

A TMA `cp.async.bulk.tensor` [21] instruction can carry a *multicast mask*: a bitmap indicating which CTAs in the issuing cluster should receive the loaded tile. The hardware performs a single GMEM read, a single L2 lookup, and writes the tile into the SMEM of every masked CTA. Each receiving CTA ends up with its own complete SMEM copy, so subsequent reads are local-SMEM fast.

Multicast is the standard cluster-broadcast primitive in CUTLASS GEMM [28]: when the cluster is partitioned along the M dimension and all CTAs need the same B tile (or vice versa), multicast reduces the GMEM/L2 traffic by a factor equal to the multicast degree. Two operational details matter for the characterization. First, the multicast issue must come from one elected leader CTA; the other receivers wait on an mbarrier. Second, multicast still costs SMEM proportional to the multicast degree (one full copy per receiver), which competes with the SMEM budget available for pipeline stages. Community reports [24] of multicast underperforming independent unicast TMAs are consistent with the issue serializing on the leader and with the L2 already absorbing duplicate reads in the non-multicast case; Section 4 isolates the regimes in which this holds.

2.6. Distributed shared memory (DSM)

DSM exposes the SMEM of every CTA in a cluster as a single distributed address space [11, 16]. A thread in CTA i can issue an ordinary SMEM load or store whose address falls inside CTA j's SMEM partition; the request travels over the cluster crossbar to the remote SM and back. Latency is higher than local SMEM access but substantially lower than going through L2 [11, 12].

DSM and TMA multicast solve the same logical problem (getting one GMEM tile to N cluster SMs) through different hardware paths. With DSM, one CTA loads the tile into its local SMEM (typically via TMA), and sibling CTAs read the tile remotely over the crossbar; only one SMEM copy exists. With multicast, every receiving CTA holds its own copy. The tradeoff is therefore SMEM capacity (DSM wins, by a factor of the cluster size) against per-access latency and bandwidth (DSM loses on each remote read, but the loss is constant per access), and the crossover depends on the *reuse count*: how many times each consumer re-reads the tile. CUTLASS [28] chose multicast for operand broadcast in its GEMM mainloops, where reuse is high; production attention kernels have at times reached for DSM in low-reuse, SMEM-constrained regimes. Section 4 quantifies when each choice is correct.

3. Experimental Methodology

This section describes the hardware platforms, software stack, workload corpus, microbenchmark harness, and measurement protocol used to produce the results in Section 4.

3.1. Hardware platforms

We evaluate on NVIDIA H100 SXM5 (SM90, Hopper): 132 SMs, 50 MB L2, 80 GB HBM3 at 3.35 TB/s peak, 228 KB SMEM per SM, cluster size up to 16 CTAs, single TMA engine per SM [16]. Each measurement is preceded by a warmup that reaches a steady thermal state. Section 5

extends the CUTLASS GEMM sweep to server Blackwell (B200, SM100); the H100 setup above is otherwise held constant.

3.2. Software stack

All measurements use CUDA Toolkit 12.6 with driver 560.35 on H100 SXM5. The production kernel in the catalog is taken from the upstream FlashAttention-3 [31] repository.

3.3. Workload corpus

FlashAttention-3 [31] forward, prefill and decode paths, is the production kernel we evaluate end-to-end. FA3 routes Q, K, and V through TMA on the contiguous-KV path, falls back to `cp.async` for paged KV, and gates the TMA-paged path on an in-tree compute-bound predicate. We extract every GMEM→SMEM load site in FA3, label the mechanism actually issued at runtime (TMA, TMA multicast, `cp.async`), and record the source-level conditions that gate the dispatch. Mechanism labels are cross-checked against SASS at the JIT-symbol level for the FA3 paged-KV dispatch, where the source-level predicate is non-trivial.

CUTLASS GEMM mainloops on SM90 dispatch TMA (and TMA multicast for operand broadcast) by default; they let us test the same TMA-vs-`cp.async` question on a non-attention workload. We benchmark 18 production GEMM shapes drawn from the projection layers of Llama-3-8B, Llama-3-70B, and DeepSeek-V2 (both dense and MLA compression projections). Per shape and datatype we sweep the dimension $M \in \{16, 64, 256, 512, 2048, 8192\}$, giving $18 \times 6 \times 3 = 324$ cells; each cell is a comparison between the TMA-dispatched CUTLASS kernel and a forced `cp.async`-only variant of the same tile schedule. For each TMA/cp.async variant, we choose the tile size that achieves the lowest runtime. Wall-time is the median of 100 timed iterations; run-to-run variation stays below 1%.

Workloads considered but not measured. We also examined FlashInfer [33] and its FlashMLA [8] companion as candidate workloads, but both currently dispatch only `cp.async` on the paged-KV path; enabling TMA-paged for an apples-to-apples comparison would require a significant rewrite of each kernel's producer warpgroup, which is beyond the scope of this characterization study.

Table 1 summarizes the attention-kernel shapes, the shared sweep axes, and the CUTLASS GEMM shapes. The three attention shapes are chosen to span the GQA-ratio axis.

3.4. Microbenchmark harness

Each hypothesis in Section 1 names a single discriminating feature (transfer size, rank, multicast degree, DSM reuse count, SMEM pressure). For each, we build a self-contained CUDA kernel that exercises only that feature and nulls out the others. Conventions:

Isolation. Microbenchmarks for multicast scaling fix the per-CTA tile shape and sweep cluster size 1–16 (H100), so that any speedup is attributable to multicast and not to a change in working-set shape.

TABLE 1: Workload corpus. The M of every GEMM shape is swept over $\{16, 64, 256, 512, 2048, 8192\}$ in each of the three datatypes, giving $18 \times 6 \times 3 = 324$ GEMM cells.

(a) Attention-kernel shapes (head dim D=D_v=128, causal)

Model tag	Query heads H	KV heads H_k	GQA ratio H/H_k	Production shape
`dense_mha`	32	32	1	Llama-3-8B [14]
`gqa`	64	8	8	Llama-3-70B [14]
`mla`	128	128	1	DeepSeek-V2 [4]

(b) Attention sweep axes

Phase (query len SQ)	decode (SQ=1), chunked (SQ=512), prefill (SQ=CTX)
Context (KV len CTX)	8K, 16K, 32K, 64K, 128K tokens
Page size	16, 32, 64, 128, 256 tokens
Datatype	BF16, FP16, FP8

(c) CUTLASS GEMM, dense projections ($N \times K$)

Family	`qkv_proj`	`o_proj`	`mlp_gate`	`mlp_down`
Llama-3-8B	6144 × 4096	4096 × 4096	14336 × 4096	4096 × 14336
Llama-3-70B	10240 × 8192	8192 × 8192	28672 × 8192	8192 × 28672
DeepSeek-V2	7680 × 5120	5120 × 5120	12288 × 5120	5120 × 12288

(d) CUTLASS GEMM, DeepSeek-V2 MLA projections ($N \times K$)

Family	`q_down`	`kv_down`	`q_up`
DeepSeek-V2 MLA	1536 × 5120	512 × 5120	24576 × 1536
	`k_up`	`v_up`	`o_proj`
	16384 × 512	16384 × 512	5120 × 16384

Timing. Wall-clock time uses CUDA event pairs over up to 20 warmup iterations followed by up to 200 timed iterations (median reported, L2 flushed between iterations).

Counters. Per-mechanism architectural attribution uses NCU [18] counters. Counter collection is run in a separate invocation from timing measurements.

3.5. Sweep parameters

FA3 attention sweeps. Three model shapes serve as proxies for deployed configurations [4, 14]: dense MHA for Llama-3-8B, GQA for Llama-3-70B, and MLA for DeepSeek-V2 (with FlashMLA [8] cited as the production MLA reference for the page-block-size convention). For each model we sweep KV length across a factor-of-two ladder spanning the long-context regime and a short-context companion ladder, three execution phases (decode, chunked prefill, full prefill), and a page-size grid chosen to straddle the FA3 K-tile boundary for the datatypes BF16, FP16, and FP8. At every TMA-eligible page size we run an equivalent comparison against a forced `cp.async`-paged variant of the same kernel so the load mechanism is the only varied factor.

Microbenchmarks. TMA 2D throughput sweeps SMEM tile height and pipeline depth at a fixed tile width. TMA multicast sweeps cluster size, sharing-group size, and SMEM tile shape, and contrasts an explicit-multicast mode against a redundant-unicast mode at matched footprints to

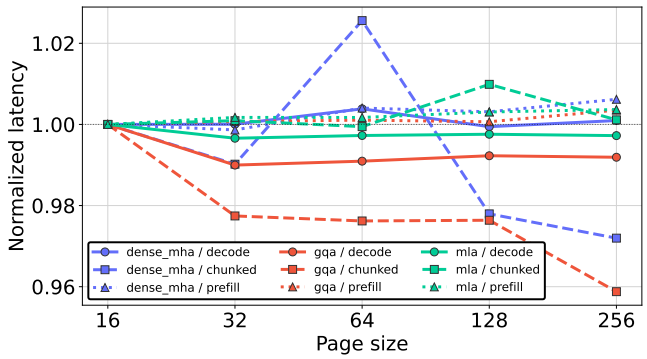


Figure 3: Median latency of the `cp.async`-paged FA3 forward at ctx=131,072, normalized per cell to the page-size-16 baseline. The nine curves span the model × phase product (dense MHA, GQA, MLA) × (decode, chunked prefill, prefill) on H100 FP16.

expose the multicast-vs-redundant-unicast crossover. Finally, a DSM broadcast microbenchmark sweeps tile sizes of transfers within a cluster.

4. Evaluation

In this section, we dive into the evaluation of various optimization choices in the context of TMA on the workloads described in Section 3.

4.1. Impact of KV page size

Figure 3 sweeps page size from 16 to 256 on the `cp.async`-paged path. All nine (model, phase) curves stay within ±4% of their page-size-16 baseline, with no systematic trend in page size. Over 50 timed iterations per point, all 36 non-baseline cells have 95% confidence intervals within 2.5% of their median. Page-size choice is therefore decoupled from kernel cost on either load path.

This decouples the page-size decision from kernel performance and pushes it entirely to the allocator. The relevant tradeoff is between tail-token waste at small page sizes and fragmentation pressure at large ones, and it is workload-specific: long-context workloads (RAG, code assistants, agent loops) keep their last page mostly full, so the fragmentation cost of larger page sizes is minor, and they should adopt page sizes at the TMA-dispatch threshold to unlock the mechanism gain reported in the next section. Short-context, high-concurrency workloads (chat, function calling) keep many sequences whose last page is mostly empty, and the per-sequence wasted tokens dominate the memory budget; for these, small page sizes remain the right choice.

> ***Observation #1: Page size is kernel-performance-neutral, but workload-specific.*** *Forward-pass latency at fixed context is independent of page size on both load paths to within noise. The choice is an allocator-level tradeoff between fragmentation and TMA-dispatch eligibility, and it should be made per workload: large pages for long-context serving, small pages for short bursty traffic.*

4.2. TMA vs. `cp.async` crossover point

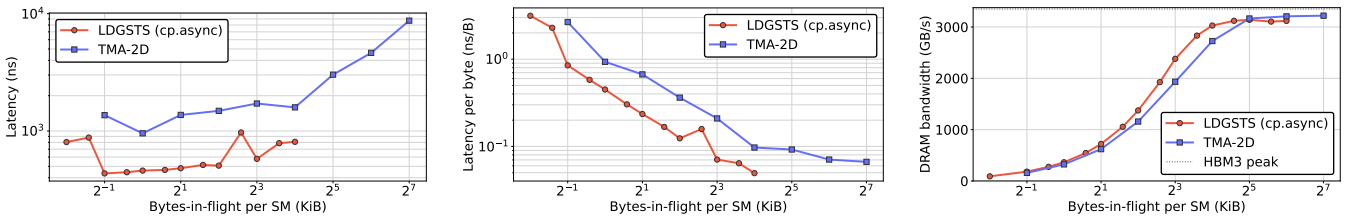
To pin down the mechanism behind the production-kernel results, we strip the comparison to its microbench primitive: one LDGSTS issue per thread, or one 2D TMA per CTA. The throughput companion sweeps the saturation curve by pipelining many in-flight loads. Three findings emerge, summarized in Figure 4.

TMA carries a fixed per-issue tax. In panel (a), at the smallest matched operating point of 0.5 KiB bytes-in-flight per SM, TMA's median single-shot latency is 1.4 μs versus LDGSTS's 0.43 μs, a 3.1× tax. The gap narrows but does not close as bytes-in-flight grow: at 1 KiB it is 2.1×, at 2–8 KiB it stays in the 2.8–3.0× range, and only at 16 KiB does TMA come within 2× of LDGSTS. The gap is the descriptor walk, bulk-copy issue, and synchronization completion machinery that TMA carries unconditionally. In panels (a) and (b), LDGSTS latency jitters by ∼10% at the smallest sizes, where the measurement sits near the noise floor; this jitter is still far below the 2–3× latency gap TMA carries in the same range.

The fixed tax amortizes hard. Panel (b) shows ns/byte collapsing by ∼40× over the swept range for *both* mechanisms, with LDGSTS holding a 2–3× per-byte advantage across the 0.5–16 KiB range. As bytes-in-flight grow, both curves converge toward the same per-byte floor: TMA reaches ∼0.07 ns/B at 128 KiB, statistically indistinguishable from LDGSTS's ∼0.05 ns/B at its 16 KiB ceiling. The descriptor cost is genuinely a fixed overhead, not a per-byte one, so it disappears as soon as the kernel keeps enough bytes in flight per issue.

The HBM3 ceiling is reached by both, but TMA crosses it without pipelining. Panel (c) shows that LDGSTS gets to ∼3.1 TB/s (94% of HBM3 peak) at 32 KiB Bytes in Flight (BIF), but only by stacking 4 CTAs/SM×stages×256 threads×float4 to fabricate that many in-flight bytes. TMA reaches a slightly higher ∼3.2 TB/s (96%) at 32 KiB BIF using a single CTA's single descriptor with a few pipeline stages. *The crossover point at which TMA becomes the right choice is therefore not a throughput point but an architectural one*: when the per-CTA work cannot mask the LDGSTS issue stream, or when a single tile exceeds what 256 threads can address with one float4 LDGSTS each (∼16 KiB), TMA is the only path that delivers the bandwidth in one issue.

> ***Observation #2: TMA wins only past a per-issue payload threshold.*** *TMA's latency is 2–3× LDGSTS's at the smallest payloads where both mechanisms are measurable (0.5–8 KiB bytes-in-flight per SM), converges within 2× at 16 KiB, and reaches statistical parity in per-byte cost at ≥32 KiB. Two distinct crossover thresholds matter for kernel authors: the* per-byte cost *curves meet near 16 KiB, and the* sustained throughput *curves saturate the same HBM3 ceiling at ∼32 KiB. Below 16 KiB bytes-in-*



(a) Latency for data transfer size. (b) Latency per byte (cost-amortization view). (c) Sustained throughput.

Figure 4: Latency and throughput microbenchmarks of LDGSTS (cp.async) and TMA-2D on H100 SXM5, with both mechanisms plotted against bytes-in-flight per SM. *(a)* median round-trip latency of a single load. *(b)* Latency per byte, isolating fixed per-issue cost from per-byte cost. *(c)* max sustained DRAM bandwidth across the config sweep; dotted line marks the H100 HBM3 advertised peak.

flight per SM, cp.async *is faster end-to-end; above* 32 *KiB, TMA saturates the same DRAM ceiling at a fraction of the issue-side machinery, leaving more warps free for compute.*

4.3. TMA vs. cp.async in production kernels

Figure 5 sweeps the TMA-vs-cp.async speedup over the model, phase, datatype, and KV-length axes at the TMA-eligible page size. The pattern is sharply phase-dependent. *Decode* cells sit on the zero line: the kernel is HBM-bandwidth-bound and both load paths move the same bytes from DRAM, so the TMA descriptor-and-tile machinery has nothing to amortize. *Chunked prefill* and *prefill* cells, by contrast, are compute-bound and TMA delivers a 5.7–11.9% speedup in BF16/FP16 and a much larger 13.4–22.2% in FP8. At 95% confidence the prefill mean speedups are +7.9% [+6.9, +8.9] (BF16), +8.2% [+7.6, +8.8] (FP16), and +16.5% [+15.5, +17.5] (FP8), and every prefill cell is individually positive; decode ties zero, with intervals of [−0.18%, +0.22%] (BF16), [−0.17%, +0.16%] (FP16), and [−0.72%, +0.12%] (FP8). The bump in speedup due to TMA stems from a more efficient use of warp issue bandwidth. TMA keeps memory accesses in flight using fewer issue slots, freeing the rest for compute. The extra FP8 headroom traces to FA3's tile scheduler: at FP8/hd=128 it picks a larger tile size (192) in the prefill mainloop (vs. 128 in BF16/FP16), so each TMA issue moves a 50% larger tile and the per-issue overhead is amortized harder, while the smaller bytes-per-token of FP8 lets compute keep up at the wider tile.

Under-the-hood: instruction-level mechanism. Table 2 contrasts NCU counters for TMA-paged against cp.async-paged on three canonical FA3 operating points spanning the compute-to-memory-bound spectrum: *TC-bound* at dense_mha/prefill/FP16/ctx=16 K (79% tensor pipe, 5.7% of HBM peak), *Balanced* at MLA/chunked/FP8/ctx=7680 (compute and memory both with headroom), and *HBM-bound* at dense_mha/decode/FP16/ctx=128 K (92% of HBM3 peak). Three observations tie back to the Fig 5 pattern. *First*, TMA and cp.async move identical DRAM bytes and issue identical WGMMA counts in every regime, so the mechanism difference is not about data movement but about the instruction stream that surrounds it: TMA replaces per-thread LDGSTS with bulk TMA-pipe issues at a consistent 26–33× compression across all three regimes, and hoists per-thread address arithmetic out of the mainloop (FMA and ALU pipes drop 8–22%). *Second*, wall-time payoff tracks which pipe is on the critical path: at TC-bound and Balanced, the freed warp-scheduler bandwidth cashes out as +7.8 pp higher tensor-pipe utilization in both and 10–21% shorter GPC cycles. At HBM-bound decode, DRAM is at 92% of HBM3 peak in both arms, so mechanism choice cannot buy any wall time even though the same 15×+ instruction-count compression is present. *Third*, this is why FA3's runtime dispatch heuristic disables TMA-paged at decode by default: the instruction-side compression is still delivered when TMA is dispatched, but it produces no wall-time change when DRAM is on the critical path (HBM-bound column).

4.3.1. Generalization to CUTLASS GEMM. To test whether the mechanism generalizes beyond attention, we repeat the TMA-vs-cp.async comparison on 324 CUTLASS SM90 GEMM cells (18 shapes × 6 M-values × 3 datatypes, listed in Table 1). TMA wins in 319 of 324 cells at a median 1.24× speedup, driven by the same memory-issue compression seen in the FA3 counter table (30–48× across GEMM regimes). The largest wins (up to 1.31×) occur where the cp.async arm is SM-utilization-limited by its issue stream; wins shrink toward 1.00× as GEMMs approach the HBM ceiling, matching the FA3 decode ceiling. The 5 losing cells are all at the smallest M ($M \in \{16, 64\}$) where too few bytes-in-flight per SM prevent the descriptor overhead from amortizing, and TMA is 2–3% slower—the same crossover threshold observed in the Fig 4 microbenchmarks. The CUTLASS result confirms that the TMA dispatch rules derived from the FA3 catalog generalize to the operand-broadcast GEMM workload without qualitative change.

Observation #3: The TMA advantage is a single mechanism with regime-dependent payoff. *Across FA3 attention and CUTLASS GEMM, TMA replaces per-thread memory issues with bulk hardware issues at an order-of-magnitude compression in every regime tested; what varies is whether that compression converts to wall time savings,*

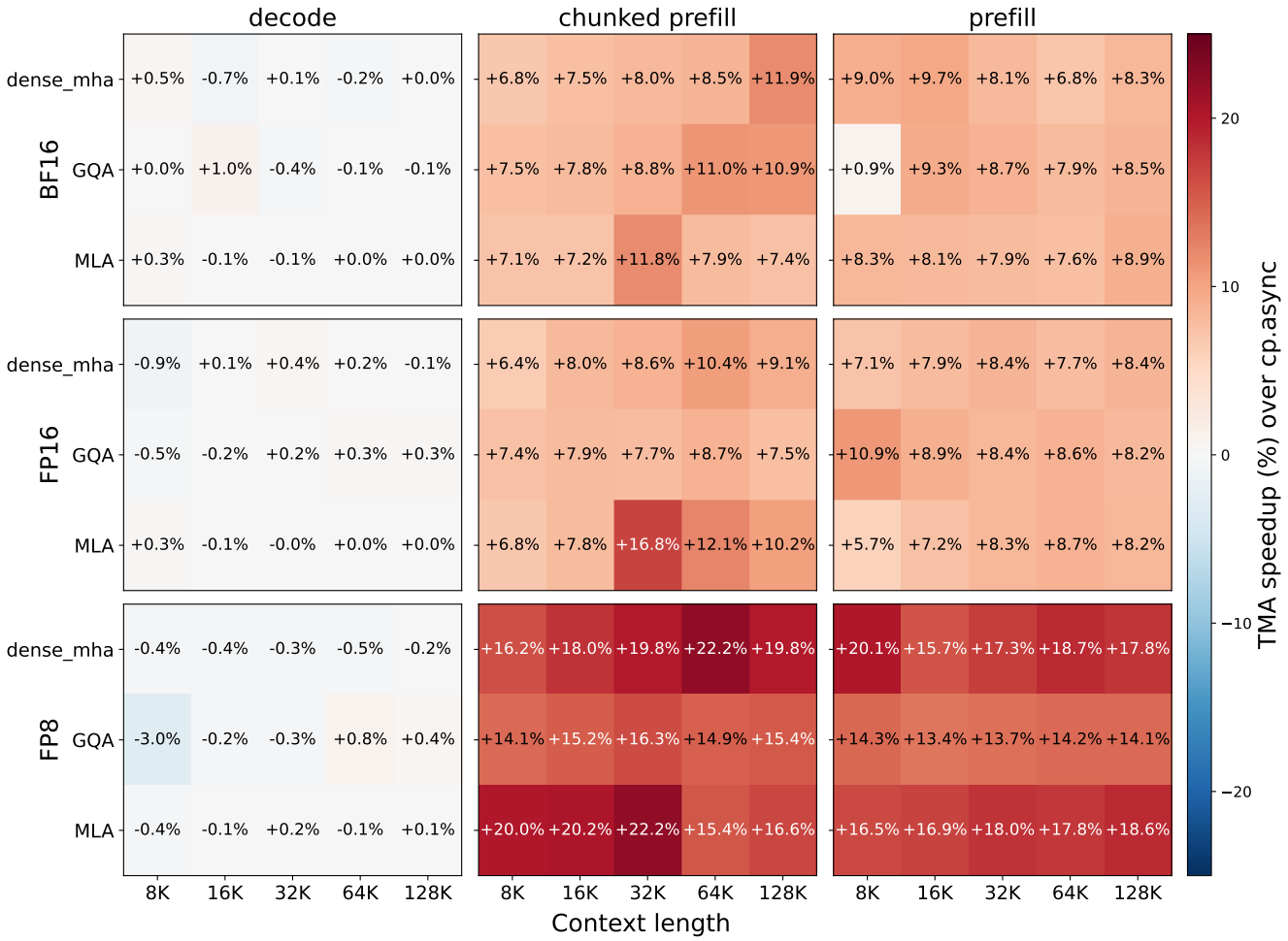


Figure 5: TMA-paged speedup over forced `cp.async`-paged on the FA3 forward at the smallest TMA-eligible page size per datatype (BF16 and FP16: ps=128; FP8: ps=192). Each cell reports $1 - t_{\text{TMA}}/t_{\text{cp.async}}$ as a percentage on H100, causal, across the three model shapes (dense MHA, GQA, MLA), the three execution phases (decode, chunked prefill, prefill), and the KV-length ladder $\{8\text{K}, 16\text{K}, 32\text{K}, 64\text{K}, 128\text{K}\}$.

which depends on the critical path. At decode, attention is HBM-bandwidth-bound and TMA ties `cp.async` *to within noise. At chunked prefill and prefill, which are compute-bound, TMA delivers a 5.7–11.9% wall-clock win in BF16/FP16 and 13.4–22.2% in FP8. The win grows with the per-tile work the datatype lets the scheduler pack: FP8 gains extra ground because FA3 selects a 50% wider tile, amortizing the descriptor and per-issue overhead over more bytes per load. Kernels that straddle the crossover (small tiles at small M) can instead pay a small penalty when the descriptor cost fails to amortize.*

4.4. TMA multicast vs redundant unicast

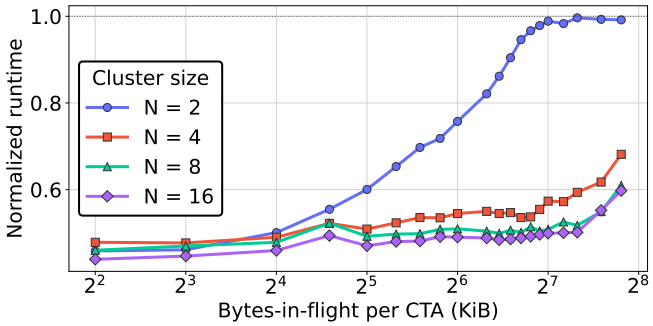
When the same tile is read by multiple CTAs, Hopper offers two paths to collapse the duplication: *(i)* let each CTA issue its own unicast TMA and rely on the GPC's LRC to deduplicate the reads (redundant unicast), or *(ii)* have the leader CTA issue a single multicast TMA whose hardware fan-out delivers the tile into all peer SMEMs (TMA multicast). The two mechanisms move identical bytes from DRAM, but they expose very different upstream traffic and very different synchronization requirements.

Figure 6 sweeps both arms across tile size and cluster size N, normalizing the redundant unicast arm to the TMA multicast. Panel (a) shows the normalized kernel runtime: redundant unicast is $\mathbf{0.45}-\mathbf{0.79}\times$ the multicast latency at every $N \geq 2$, a $1.3-2.2\times$ speedup that grows with N and saturates near $2\times$ at $N \geq 8$. The win is not the load itself but its absence of coordination: the multicast protocol requires a cluster-level synchronization which sits on the critical path. Redundant unicast uses per-CTA-local mbarriers and runs free.

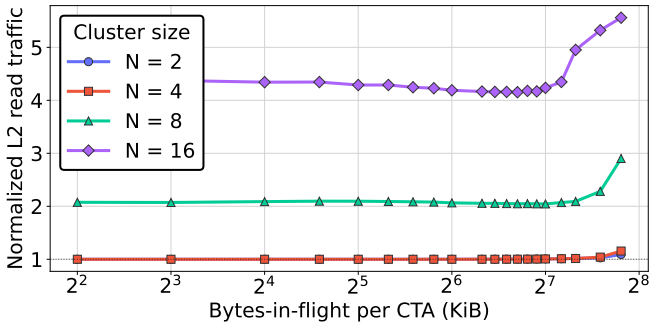
Panel (b) shows the normalized L2 read-sector traffic. At $N \leq 4$ the two arms tie within 1%: the LRC's coalescing window comfortably absorbs the inter-CTA arrival skew, and L2 sees a single sector per tile regardless of which path issued the loads. At $N = 8$ redundant unicast leaks

TABLE 2: PER-KERNEL NCU COUNTERS COMPARING TMA AGAINST `cp.async` ON THREE REPRESENTATIVE FA3 FORWARD REGIMES. BOLDED ROWS IDENTIFY THE MECHANISM-DISTINGUISHING COUNTERS AND WALL TIME.

	TC-bound (prefill/16K/FP16)			**Balanced** (chunked/8K/FP8)			**HBM-bound** (decode/128K/FP16)		
Metric	**TMA**	`cp.async`	**ratio**	**TMA**	`cp.async`	**ratio**	**TMA**	`cp.async`	**ratio**
Tensor pipe (% peak)	**79.3%**	71.5%	**1.11**×	37.9%	30.1%	**1.26**×	25.4%	25.5%	1.00×
SM active (% peak)	99.5%	99.5%	1.00×	95.1%	95.5%	1.00×	91.0%	91.3%	1.00×
DRAM bytes	689 M	690 M	1.00×	327 M	327 M	1.00×	2.15 G	2.15 G	1.00×
DRAM throughput (%)	5.7%	5.2%	1.10×	22.5%	18.0%	1.25×	**92.1%**	**92.4%**	1.00×
L2 hit rate (%)	94.2%	94.0%	1.00×	69.2%	73.5%	0.94×	1.5%	1.5%	1.00×
Total inst issued	**1.09 G**	**1.26 G**	**0.86**×	**145.6 M**	**166.0 M**	**0.88**×	**66.5 M**	**87.4 M**	**0.76**×
WGMMA	33.8 M	33.8 M	1.00×	1.6 M	1.6 M	1.00×	2.1 M	2.1 M	1.00×
Memory	**1.3 M**	**34.1 M**	**0.04**×	**57 K**	**1.9 M**	**0.03**×	**131 K**	**4.2 M**	**0.03**×
FMA pipe	451 M	506 M	0.89×	66 M	72 M	0.92×	29 M	35 M	0.83×
ALU pipe	260 M	291 M	0.89×	36 M	39 M	0.91×	16 M	21 M	0.78×
GPC cycles (wall time)	**5.17 M**	**5.73 M**	**0.90**×	**617 K**	**778 K**	**0.79**×	**993 K**	**993 K**	**1.00**×



(a) Kernel runtime for redundant unicast normalized to TMA multicast (< 1 means unicast is faster).



(b) L2 read sectors served for redundant unicast normalized to TMA multicast (> 1 means unicast suffers more L2 traffic).

Figure 6: Normalized runtime and L2 reads for redundant unicast vs. explicit TMA multicast on H100. Each cluster of N CTAs co-located in a thread-block cluster reads the same tile per iteration; the redundant unicast arm has every CTA issue its own unicast TMA and lets the GPC L2 Request Coalescer (LRC) merge duplicates; the explicit arm uses TMA multicast from the leader CTA.

2.1× more L2 sectors; at $N = 16$, **4.4**× more. The skew exceeds the LRC's tracking depth, misses deduplication opportunities, and the request bandwidth at L2 scales linearly with N on the redundant unicast path while TMA multicast holds a constant 1 sector per tile per iteration by construction. Table 3 summarizes the decision rule for choosing between explicit TMA multicast and redundant unicast on Hopper.

Observation #4: TMA multicast trades runtime for L2 cleanliness. *Letting the LRC deduplicate N CTAs' independent TMA issues is up to 2× faster end-to-end than explicit multicast because the cluster sync that multicast requires dominates the critical path. The cost is L2 request bandwidth: at cluster ≥ 8, implicit dedup spends 2–4× more L2 sectors per tile because the inter-CTA skew breaks LRC coalescing. Choose explicit multicast when the surrounding kernel is L2-bandwidth-bound or reuses tiles across passes (e.g., CUTLASS GEMM operand broadcast); choose redundant unicast when the workload is streaming and L2 is a pass-through to DRAM.*

4.5. TMA multicast vs DSM-based broadcast

Hopper exposes two hardware primitives that solve the same logical problem: get one GMEM tile resident in the SMEM of N CTAs of a thread-block cluster. *TMA multicast* accomplishes this with a single `cp.async.bulk.tensor.-multicast::cluster` issue from a leader CTA; the TMA engine pulls the tile once from L2 and uses the SM-to-SM crossbar to fan it out, pushing it into each peer's SMEM and signaling each peer's `mbarrier`. *DSM-based broadcast* accomplishes the same end state with two software steps: the leader issues a unicast TMA that lands the tile in its own SMEM, after which peer CTAs read the tile through the same SM-to-SM crossbar via `ld.shared::cluster` instructions and potentially stage it into their own SMEM. Both protocols move the same bytes from DRAM to N peer SMEMs and pay one DRAM trip per tile; they differ only in how the cross-cluster step is executed: *push*, hardware

TABLE 3: Workload-specific decision rule for choosing between explicit TMA multicast and redundant unicast on Hopper.

Condition	Recommended primitive
Cluster $N = 2$ AND bytes-in-flight per CTA $\gtrsim$ 100 KiB (largest tiles in the sweep, Fig. 6(a))	Either path; ratio approaches 1.0 and L2 traffic is tied. The extra implementation complexity of multicast is not justified.
Cluster $N \leq 4$ at smaller tiles, OR any $N \geq 8$ where the kernel is streaming / not L2-bound	**Redundant unicast** Redundant unicast is 1.3−2.1× faster on runtime because the cluster-level synchronization required by the TMA multicast protocol dominates the critical path; the extra L2 sectors that LRC leaks at $N \geq 8$ are absorbed without cost when L2 is just a pass-through to DRAM.
Cluster $N \geq 8$ AND kernel is L2-bandwidth-bound or reuses tiles across passes (*e.g.*, CUTLASS GEMM operand broadcast)	**Explicit TMA multicast.** The 2−4× L2 sector savings (Fig. 6(b)) translate to wall-time gain once L2 is on the critical path, outweighing the sync overhead.

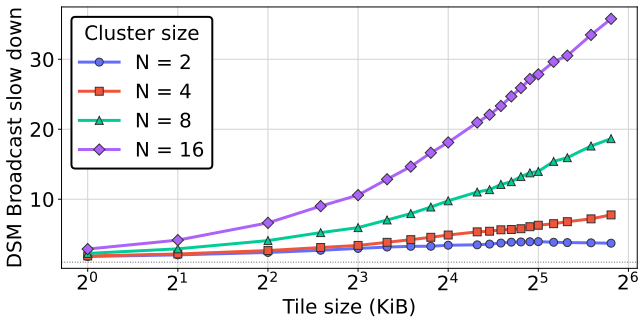


Figure 7: Normalized slowdown of DSM broadcast with respect to TMA multicast at various cluster sizes and tile sizes. Ratio > 1 means TMA multicast is faster.

driven, single issue (multicast); or *pull*, software driven, $N-1$ parallel issues bracketed by a cluster-level synchronization primitive on each side (DSM).

Multicast issues a single multicast TMA from the leader. DSM issues a unicast TMA on the leader, performs a cluster-wide barrier, has peers pull the tile via cluster-scope shared-memory loads and stage it into their local SMEM. We sweep cluster size $N \in \{2, 4, 8, 16\}$ and 20 tile sizes from 1 KiB to 56 KiB. Both protocols incur the same DRAM cost per tile, so any wall-time delta is purely the cross-cluster fan-out mechanism.

Figure 7 shows the slowdown of DSM broadcast with respect to TMA multicast. TMA multicast is uniformly faster at every cell, and the gap widens monotonically along both axes. At N=2 the wall-time ratio sits in the 1.8−3.9× range and is essentially flat in tile size; at N=4 it grows from 1.9× at the smallest tile to 7.8× at the largest; at N=8 from 2.3 to **18.6**×; and at N=16 from 2.9 to **35.8**×. Two scaling regimes explain the shape. *First*, the small-tile floor of ∼2−3× across all N is the structural overhead of the DSM protocol's higher synchronization overhead. At small tiles, where each iteration's data movement is short, this synchronization tax dominates. *Second*, the large-tile divergence reflects the asymmetric crossbar cost: DSM moves $(N-1) \times$ tile_bytes over the SM-to-SM crossbar via $(N-1)$ independent software-driven pull streams, while multicast issues a single hardware-driven fan-out from a single L2 fetch. DSM's cross-cluster traffic scales linearly with $N-1$ and with tile size; multicast's amortizes both factors into one issue. The ∼36× corner at (N=16, 56 KiB) is the consequence of both effects multiplying.

> ***Observation #5: TMA multicast dominates DSM broadcast on Hopper when the broadcast pattern is regular.*** *TMA multicast is* 1.8× *to* 35× *faster than a DSM-based broadcast across all measured cluster sizes and tile sizes, with the gap growing in both. The mechanism gap has two components: a fixed* ∼2−3× *synchronization-overhead floor, and a linear-in-*$(N-1)$ *crossbar-traffic penalty that dominates at large tiles. DSM remains the right primitive when (i) the data is already SMEM-resident (no DRAM trip to amortize), (ii) the consumer set is irregular and not addressable as a single multicast mask, or (iii) SMEM capacity dictates a single shared copy rather than N replicas. For the regular broadcast pattern that CUTLASS and FA3 actually use (operand tile or KV tile to every cluster CTA), TMA multicast is the correct hardware choice on Hopper.*

5. Extension to Blackwell (SM100)

To test whether the H100 dispatch rules generalize, we repeat the study on CUTLASS GEMMs (Sec. 4.3.1 Table 1) on the Blackwell (SM100) B200. B200 retains the TMA, cluster, multicast, and DSM mechanisms of Hopper but adds Tensor Memory (TMEM), which moves the UMMA accumulator state out of SMEM (freeing ∼256 KB per SM for larger TMA tiles), and a 2-CTA UMMA that changes per-issue tile requirements.

TMA wins widen on B200. TMA wins 320 of 324 cells at a median 2.12×, up from 1.24× on H100 (Sec. 4.3.1); the 4 ties sit in the small-M crossover and no cell shows a loss. Two SM100 shifts explain the widened advantage: (i) per-SM tensor throughput roughly doubles while LDGSTS retires through the unchanged per-thread issue machinery, so `cp.async` can no longer feed the tensor cores and the TMA arm's issue-side compression (Table 2) cashes out at a larger ratio; and (ii) the HBM3e bandwidth of B200 [19] outgrows kernel demand so no cell saturates HBM, emptying the DRAM-bound tie regime that existed on H100.

The reported gain is a lower bound: we use the 1-SM UMMA path to keep both arms comparable, since 2-SM UMMA is TMA-only. The qualitative dispatch rules (payload threshold, compute-bound gate, multicast preconditions) transfer to B200 without change; only the absolute crossovers shift, tracking tensor-core and bandwidth scaling rather than copy-engine improvements. Extending the analysis to production FlashAttention-4 [3] on B200 remains future work.

6. Related Work

Our characterization touches six prior threads; none resolves the comparative dispatch question we frame.

Memory-hierarchy microbenchmarking of Hopper and Blackwell. Luo et al. [12, 13] characterize the TMA, L1TEX, L2, and DSM paths on H100 in isolation, and Jarmusch et al. [6] extend that style of dissection to Blackwell; Lühnen et al. [11] report complementary measurements of the cluster crossbar. These works bound each mechanism alone, but never contrast TMA against `cp.async` on the same operand or multicast against DSM on the same broadcast workload.

Asynchronous memory copy on Ampere and Hopper. Li et al. [9, 10] characterize the `cp.async` (LDGSTS) path on A100 and H100, mapping its overlap envelope and per-instruction issue cost. Their measurements bound one side of our TMA-vs-`cp.async` comparison but never contrast the two paths directly on identical operands.

Production kernels and libraries using TMA. Production libraries encode TMA dispatch heuristics without publishing the empirical crossovers behind them. CUTLASS [28] uses TMA and multicast for GEMM operand tiles; FlashAttention-3 [27] routes Q, K, and V through TMA on prefill but gates TMA-paged-KV behind a workload-dependent fallback to `cp.async`, FlashAttention-4 [3] revisits the same dispatch on SM100, and FlashInfer [32], FlashMLA [8], and cuBLAS [17] make analogous operand-by-operand TMA-vs-fallback choices. The FA3 in-source dispatch comment [31] and the Colfax [2] and PyTorch [5] write-ups all report TMA effects without isolating the mechanism.

KV cache memory management. PagedAttention [7], vLLM [30], SGLang [34], TensorRT-LLM [23], and vAttention [26] treat page size as an allocator-level tradeoff and do not model its interaction with TMA-eligibility on FA3 and FA4. Operator-default page sizes fall back to `cp.async`-paged on H100, leaving measurable TMA bandwidth unused.

Vendor documentation and community reports. NVIDIA's Hopper Tuning Guide [20] describes the API but not crossover points. Developer-Forum reports [24, 25] document deployed anomalies that our cross-cutting case study explains via TMA-engine occupancy and leader-side issue serialization.

Prior Blackwell characterization. Jarmusch et al. [6] dissect the memory hierarchy and SM pipelines of consumer Blackwell, and Chen and Patel [1] provide per-mechanism baseline measurements for server Blackwell (B200, SM100) [19], bounding TMA, TMEM, and the 2-CTA UMMA path in isolation. Section 5 builds on these baselines to extend our workload-grounded characterization from H100 to B200.

To our knowledge, no prior work pairs a workload-grounded view of TMA dispatch choices in a production attention kernel with microbenchmarks that explain why those choices were made. Our paper distills the resulting crossovers into concrete rules of thumb for TMA and multicast dispatch on the Hopper generation of NVIDIA GPUs.

7. Conclusion

TMA is used pervasively in production Hopper kernels, yet its deployment is conspicuously selective and the heuristics that gate dispatch are workload-specific and under-documented. We closed that gap with a workload-grounded study in three stages: an operand-by-operand catalog of the TMA, multicast, DSM, and `cp.async` choices that FlashAttention-3 makes; targeted microbenchmarks on H100 that isolate the discriminating axes; and a cross-cutting paged-KV case study that exercises every rule on a deployed kernel.

The study answers the three questions we set out to address. *(Q1)* TMA outperforms `cp.async` only above a joint crossover in transfer size and descriptor reuse; outside that envelope (small transfers, low reuse), `cp.async` is the correct choice. The FA3 sweep comparing TMA-paged against `cp.async`-paged across model shapes, datatypes, and phases confirms that within either dispatch the page-size knob is performance-neutral within measurement noise; mechanism, not page size, is the load-bearing axis, and operators of vLLM, SGLang, and TensorRT-LLM can choose page size on allocator grounds and TMA dispatch eligibility (which disproportionately benefits compute-bound phases such as prefill operations). *(Q2)* TMA multicast beats the LRC-deduplicated redundant-unicast path when the kernel is L2 bandwidth bound; outside that regime, redundant unicast is competitive end-to-end and multicast's L2 cleanliness is not load-bearing. *(Q3)* Against DSM broadcast at matched DRAM cost, multicast wins under high per-tile reuse, while DSM wins under SMEM pressure, low reuse, or under exotic data transfer patterns.

Section 5 extends the CUTLASS GEMM sweep to server Blackwell (B200, SM100), where TMA wins 320 of 324 cells at median 2.12× (up from 1.24× on H100). The qualitative dispatch rules carry across generations; the absolute crossovers shift with tensor-core throughput and HBM bandwidth scaling rather than with copy-engine changes per se. As future work, we plan to extend the analysis to production attention kernels such as FlashAttention-4 [3] on B200. More broadly, we hope that deriving dispatch rules from a workload-grounded characterization, rather than from per-mechanism peak numbers alone, encourages similar studies of the asynchronous data-movement primitives that each new GPU generation introduces.

Acknowledgments

This work was supported, in part, by NSF CCF #1943379 and NSF CCF #2450447. We used Claude to write some of the scripts that drive our experiments and to polish the text. All experimental results were produced by running the kernels on real hardware, and the authors verified the generated code and reviewed all text for accuracy.

References

[1] K. Chen and D. Patel, "Dissecting Nvidia Blackwell - Tensor Cores, PTX Instructions, SASS, Floorsweep, Yield," SemiAnalysis, https://newsletter.semianalysis.com/p/dissecting-nvidia-blackwell-tensor, 2026, online; accessed 2026.

[2] Colfax Research, "CUTLASS Tutorial: Mastering the NVIDIA Tensor Memory Accelerator (TMA)," https://research.colfax-intl.com/tutorial-hopper-tma/, 2024, online; accessed 2026.

[3] T. Dao and others, "FlashAttention-4: A CuTeDSL Implementation of FlashAttention for Hopper and Blackwell," https://github.com/Dao-AILab/flash-attention/tree/main/flash_attn/cute, 2025, online; accessed 2026.

[4] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," arXiv preprint arXiv:2405.04434, 2024, online; accessed 2026.

[5] A. Hoque, L. Wright, and C.-C. Yang, "Deep Dive on the Hopper TMA Unit for FP8 GEMMs," PyTorch Blog, https://pytorch.org/blog/hopper-tma-unit/, 2024, online; accessed 2026.

[6] A. Jarmusch, N. Graddon, and S. Chandrasekaran, "Dissecting the NVIDIA Blackwell Architecture with Microbenchmarks," arXiv preprint arXiv:2507.10789, 2025, online; accessed 2026.

[7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[8] J. Li and S. Liu, "FlashMLA: Efficient Multi-head Latent Attention Kernels," https://github.com/deepseek-ai/FlashMLA, 2025, online; OSW Day 1; accessed 2026.

[9] R. Li, B. Hanindhito, S. Yadav, Q. Wu, K. Kavi, G. Mehta, N. J. Yadwadkar, and L. K. John, "Performance Implications of Pipelining the Data Transfer in CPU-GPU Heterogeneous Systems," *ACM Transactions on Architecture and Code Optimization (TACO)*, 2025.

[10] R. Li, S. Yadav, Q. Wu, K. Kavi, G. Mehta, N. J. Yadwadkar, and L. K. John, "Performance Implications of Async Memcpy and UVM: A Tale of Two Data Transfer Modes," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2023.

[11] T. J. Lühnen, T. Marschner, and S. Lal, "Benchmarking Thread Block Cluster," in *IEEE High Performance Extreme Computing Conference (HPEC)*, 2024.

[12] W. Luo, R. Fan, Z. Li, D. Du, H. Liu, Q. Wang, and X. Chu, "Dissecting the NVIDIA Hopper Architecture through Microbenchmarking and Multiple Level Analysis," arXiv preprint arXiv:2501.12084, 2025, online; accessed 2026.

[13] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang, and X. Chu, "Benchmarking and Dissecting the NVIDIA Hopper GPU Architecture," in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2024.

[14] Meta AI, "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024, online; accessed 2026.

[15] NVIDIA Corporation, "NVIDIA A100 Tensor Core GPU Architecture Whitepaper," https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf, 2020, online; accessed 2026.

[16] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture Whitepaper," https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper, 2022, online; accessed 2026.

[17] NVIDIA Corporation, "cuBLAS Library User Guide," https://docs.nvidia.com/cuda/cublas/, 2024, online; accessed 2026.

[18] NVIDIA Corporation, "Nsight Compute Profiling Guide," https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html, 2024, online; accessed 2026.

[19] NVIDIA Corporation, "NVIDIA Blackwell Architecture Technical Brief," https://resources.nvidia.com/en-us-blackwell-architecture, 2024, online; accessed 2026.

[20] NVIDIA Corporation, "NVIDIA Hopper Tuning Guide," https://docs.nvidia.com/cuda/hopper-tuning-guide/, 2024, online; accessed 2026.

[21] NVIDIA Corporation, "Parallel Thread Execution ISA: Data Movement and Conversion Instructions (`cp.async.bulk.tensor`)," https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#data-movement-and-conversion-instructions-cp-async-bulk-tensor, 2024, online; accessed 2026.

[22] NVIDIA Corporation, "Parallel Thread Execution ISA: Parallel Synchronization and Communication Instructions: `mbarrier`," https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#parallel-synchronization-and-communication-instructions-mbarrier, 2024, online; accessed 2026.

[23] NVIDIA Corporation, "TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference," https://github.com/NVIDIA/TensorRT-LLM, 2024, online; accessed 2026.

[24] NVIDIA Developer Forum, "Implementing H100 TMA multicast with `cuda::ptx::` functions but its slower than 8 independent TMA operations fetching same tile in cluster," https://forums.developer.nvidia.com/t/implementing-h100-tma-multicast-with-cuda-functions-but-its-slower-than-8-independent-tma-operations-fetching-same-tile-in-cluster/355253/, 2025, online; community report; accessed 2026.

[25] NVIDIA Developer Forum, "Oddity with TMA performance and its size of data transfer," https://forums.developer.nvidia.com/t/oddity-with-tma-performance-and-its-size-of-data-transfer/355268/, 2025, online; community report; accessed 2026.

[26] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[27] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[28] V. Thakkar, P. Ramani, C. Cecka, A. Shivam, H. Lu, E. Yan, J. Kosaian, M. Hoemmen, H. Wu, A. Kerr, M. Nicely, D. Merrill, D. Blasig, F. Qiao, P. Majcher, P. Springer, M. Hohnerbach, J. Wang, and M. Gupta, "CUTLASS: CUDA Templates for Linear Algebra Subroutines," https://github.com/NVIDIA/cutlass, 2023, online; accessed 2026.

[29] TheRainstorm, "flash-attention-3 batch decode is significantly slower than flashinfer on H20," GitHub issue #1572, https://github.com/Dao-AILab/flash-attention/issues/1572, 2025, online; accessed 2026.

[30] vLLM Project, "vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention," https://github.com/vllm-project/vllm, 2024, online; accessed 2026.

[31] vLLM Project, T. Dao, and others, "FlashAttention-3 (vLLM fork) — `get_pagedkv_tma` dispatch heuristic," https://github.com/vllm-project/flash-attention/blob/5824e6e2008271063c3229ab3e7032bd74abbbc6/hopper/flash_api.cpp#L456-L465, 2026, online; commit `5824e6e`, `hopper/flash_api.cpp`, function `get_pagedkv_tma`; accessed 2026.

[32] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," in *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys)*, 2025.

[33] Z. Ye and others, "FlashInfer: Kernel Library for LLM Serving (Source Repository)," https://github.com/flashinfer-ai/flashinfer, 2025, online; accessed 2026.

[34] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient Execution of Structured Language Model Programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.